Java 8 Features

Functional Interface

Lambda expression

Predicate Functional Interface

- The Functional Interface PREDICATE is defined in the *java.util.Function package*.

- It improves manageability of code, helps in unit-testing them separately, and contain some methods like.

1. Boolean test(T t) : Evaluates this predicate on the given argument.
2. static Predicate isEqual(Object targetRef)
3. default Predicate and(Predicate other)
4. default Predicate negate()
5. default Predicate or(Predicate other)

```
import java.util.function.Predicate;

public class PredicateDemo {

public static void main(String[] args)
    {
        Predicate<Integer> lesserthan = i -> (i < 18);

        System.out.println(lesserthan.test(10));
    }
}
```

Consumer Functional Interface

- Java8 landed with new feature so called functional Interface.
- Common example of such an operation is *printing* where an object is taken as input to the printing function.
- No need to define a new functional interface every time.
- Method:: void accept(T t)
- Default method: andThen()
- takes as input another instance of Consumer interface and returns as a result a *new* consumer interface

Pre-defined Consumer interface

```
@FunctionalInterface
public interface Consumer<T> {
    void accept(T t);
    default Consumer<T> andThen(Consumer<? super T> after) {
        Objects.requireNonNull(after);
        return (T t) -> { accept(t); after.accept(t); };
    }
}
```

```
import java.util.ArrayList;
import java.util.Arrays;
import java.util.function.Consumer;
public class ConsumerFunctionExample{
  public static void main(String args[]){
  Integer arr[]={10,20,70,50,60};
  ArrayList<Integer> list=new ArrayList<>(Arrays.asList(arr));
  Consumer<Integer> consumer= i-> System.out.print(" "+i);
  printList(list, consumer);
 }
private static void printList(ArrayList<Integer> list, Consumer<Integer>
    consumer) {
for(Integer element:list){
    consumer.accept(element);

 } } }
```

Demo:: andThen()

```
package com;
import java.util.ArrayList;
import java.util.Arrays;
import java.util.function.Consumer;

public class ConsumerFunctionExample2 {

public static void main(String[] args) {
 Integer arr[]={10,20,70,50,60};
  ArrayList<Integer> list=new ArrayList<>(Arrays.asList(arr));

  Consumer<Integer> consumer= i-> System.out.println(" "+i);
  Consumer<Integer> myconsumer=consumer.andThen( i-> System.out.println("Printed: "+i));
  printList(list, myconsumer);
}
private static void printList(ArrayList<Integer> list, Consumer<Integer> consumer) {
for(Integer element:list){
    consumer.accept(element);
  }
}
}
```

Java Stream API

- Stream API is used to process collections of objects.

The features of Java stream are

1. A stream is not a data structure instead it takes input from the Collections, Arrays or I/O channels.

2. Streams don’t change the original data structure, they only provide the result as per the pipelined methods.

3. Each intermediate operation is lazily executed and returns a stream as a result, hence various **intermediate operations** can be pipelined. **Terminal operations** mark the end of the stream and return the result.

- **Intermediate Operations:**

1. map: The map method is used to returns a stream consisting of the results of applying the given function to the elements of this stream.

```
List number = Arrays.asList(2,3,4,5);
List square = number.stream().map(x>x*x).collect(Collectors.toList());
```

2. filter: The filter method is used to select elements as per the Predicate passed as argument.

```
List names = Arrays.asList("Reflection","Collection","Stream");
List result = names.stream().filter(s ->s.startsWith("S"))
                .collect(Collectors.toList());
```

3. sorted: The sorted method is used to sort the stream.

```
List names = Arrays.asList("Reflection","Collection","Stream");
List result = names.stream().sorted().collect(Collectors.toList());
```

2. Terminal Operations:

- **collect:** The collect method is used to return the result of the intermediate operations performed on the stream.

```
List number = Arrays.asList(2,3,4,5,3);
Set square = number.stream()
                .map(x->x*x)
                .collect(Collectors.toSet());
```

- **forEach:** The forEach method is used to iterate through every element of the stream.

```
List number = Arrays.asList(2,3,4,5);
                number.stream().
                .map(x->x*x)
                .forEach(y->System.out.println(y));
```

- **reduce:** The reduce method is used to reduce the elements of a stream to a single value.
  The reduce method takes a BinaryOperator as a parameter.

```
List number = Arrays.asList(2,3,4,5);

int even = number.stream().filter(x->x%2==0).reduce(0,(ans,i)-> ans+i);
```

Cont…

```
package test;

import java.util.Arrays;

public class Test {

public static void main(String[] args) {
int[] numbers = {1, 2, 3, 4, 5};

int sum = Arrays.stream(numbers).reduce(0, (a, b) -> a + b);    //initial value
int sum2 = Arrays.stream(numbers).reduce(0, Integer::sum);

int sum3 = Arrays.stream(numbers).reduce(0, (a, b) -> a - b);
int sum4 = Arrays.stream(numbers).reduce(0, (a, b) -> a * b);   // 0, initial is 0, 0 * whatever = 0
int sum5 = Arrays.stream(numbers).reduce(0, (a, b) -> a / b);   // 0

System.out.println(sum);
System.out.println(sum2);
System.out.println(sum3);
System.out.println(sum4);
System.out.println(sum2);
}}
```

forEach of Inerrable Interface

- Check

```
default void forEach(Consumer<? super T> action)
 {
     Objects.requireNonNull(action);
     for (T t : this)
        { action.accept(t);
        }
}
```

Note:: Objects is in built class.

requireNonNull(T) checks for non null value.

```
package com;

import java.util.ArrayList;
import java.util.Arrays;

public class ForEachDemo {

public static void main(String[] args) {
Integer arr[] = { 10, 20, 70, 50, 60 };
ArrayList<Integer> list = new ArrayList<>(Arrays.asList(arr));

list.forEach(element-> System.out.println(element));
}
}
```

```
//a simple program to demonstrate the use of stream in java
import java.util.*;
import java.util.stream.*;

public class StreamDemo
{
  public static void main(String args[])
  {
    // create a list of integers
    List<Integer> number = Arrays.asList(2,3,4,5);
    // demonstration of map method
    List<Integer> square = number.stream().map(x -> x*x).
                          collect(Collectors.toList());
    System.out.println(square);
    // create a list of String
    List<String> names =
              Arrays.asList("Reflection","Collection","Stream");
    // demonstration of filter method
    List<String> result = names.stream().filter(s->s.startsWith("S")).
                         collect(Collectors.toList());
    System.out.println(result);
```

```
    // demonstration of sorted method
    List<String> show =
            names.stream().sorted().collect(Collectors.toList());
    System.out.println(show);
// create a list of integers
    List<Integer> numbers = Arrays.asList(2,3,4,5,2);
    // collect method returns a set
    Set<Integer> squareSet =
        numbers.stream().map(x->x*x).collect(Collectors.toSet());
    System.out.println(squareSet);

    // demonstration of forEach method
    number.stream().map(x->x*x).forEach(y->System.out.println(y));

    // demonstration of reduce method
    int even =
      number.stream().filter(x->x%2==0).reduce(0,(ans,i)-> ans+i);
    System.out.println(even);
  }
}
```

Supplier Functional Interface

```
package java.util.function;
@FunctionalInterface
public interface Supplier<T> {
    /**
     * Gets a result.
     *
     * @return a result
     */
    T get();
}
```

```
package predicate1;

import java.util.function.Supplier;

public class SupplierDemo {

 public static void main(String[] args) {

  Supplier<String> supplier= ()-> "Ranjit";

  System.out.println(supplier.get());
 }
}
```

```
public class Student {
private int roll;
String name;
public Student(int roll,String name)
{
this.roll=roll;
this.name=name;
}
public int getRoll() {
return roll;
}
public String getName() {
return name;
}
@Override
public String toString() {
return roll+" "+name;
}
}
```

```
import java.util.function.Supplier;

public class SupplierDemo2 {

public static void main(String[] args) {

Supplier studentSupplier = () -> new Student(101,"AAA");
Student student = (Student) studentSupplier.get();
System.out.println(student);
}

}
```

```
import java.util.Random;
import java.util.function.Supplier;
import java.util.stream.Stream;

public class Java8SupplierExample {
    public static void main(String[] args) {
            Supplier<Integer> randomNumbersSupp=() -> new
    Random().nextInt(100);
      Stream.generate(randomNumbersSupp)
                        .limit(5)
                        .forEach(System.out::println);
    }
}
```

Sequential Stream and Parallel Stream

```
import java.util.Arrays;
import java.util.List;

public class Main {
  public static void main(String[] args) {
    List<Integer> list=Arrays.asList(1,2,3,4,5,6,7,8,9);
    list.stream().forEach(System.out::println);
    System.out.println();
    long start = System.currentTimeMillis();
    list.parallelStream().forEach(System.out::println);
    long end = System.currentTimeMillis();
    System.out.println("Total: "+(end-start));

    start = System.currentTimeMillis();
    list.stream().forEach(System.out::println);
    end = System.currentTimeMillis();
    System.out.println("Total: "+(end-start));
  } }
```

Date Time API

- There have been several problems with the existing date and time related classes in java.

1. Date Class in both java.util as well as java.sql packages.

2. There are no clearly defined classes for time, timestamp, formatting and parsing. e.g.SimpleDateFormate

3. All the Date classes are mutable, so they are **not thread safe**. It's one of the biggest problem with Java Date and Calendar classes.

4. Date class doesn't provide internationalization, there is no timezone support. So java.util.Calendar and java.util.TimeZone classes were introduced, but they also have all the problems listed above.

Java 8 Date Time API Packages

- **java.time Package**:

  LocalDate, LocalTime, LocalDateTime, Instant, Period, Duration .

  All of these classes are immutable and thread safe.

- **java.time.chrono Package**: This package defines generic APIs for non ISO calendar systems. We can extend AbstractChronology class to create our own calendar system.

- **java.time.format Package**: This package contains classes used for formatting and parsing date time objects. Most of the times, we would not be directly using them because principle classes in java.time package provide formatting and parsing methods.

- **java.time.temporal Package**: This package contains temporal objects and we can use it for find out specific date or time related to date/time object.

- **java.time.zone Package**: This package contains classes for supporting different time zones and their rules.